



Chuyên đề: Kỹ thuật Xử lý Giá trị Tuyệt đối & Khoảng cách Manhattan

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Biên soạn: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 Duc-Minh Vu. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

1. Tổng quan & Bản đồ Tư duy (Mindmap)

Trong lập trình thi đấu (Competitive Programming), dấu giá trị tuyệt đối $|x|$ xuất hiện với tần suất rất cao trong các bài toán hình học tính toán (Computational Geometry), tối ưu hóa hàm mục tiêu (Optimization), quy hoạch động (DP) và hình học rời rạc trên lưới (Grid/Manhattan Distance). Bản chất phi tuyến (non-linear) và tính phân nhánh của hàm trị tuyệt đối thường gây cản trở cho các thuật toán tìm kiếm nghiệm trực tiếp nếu không biết cách **khử dấu trị tuyệt đối** hoặc chuyển đổi về các dạng lồi (convex) đơn giản hơn.



2. Nền tảng Lý thuyết & Tính chất Cốt lõi

2.1. Định nghĩa & Ý nghĩa Hình học

Với mọi số thực $x \in \mathbb{R}$, giá trị tuyệt đối được định nghĩa bởi:

$$|x| = \begin{cases} x & \text{nếu } x \geq 0 \\ -x & \text{nếu } x < 0 \end{cases}$$

- Biểu diễn cực đại (Max Representation):**

$$|x| = \max(x, -x)$$

Mở rộng cho hiệu của hai số:

$$|a - b| = \max(a - b, b - a)$$

💡 Mẹo / Tối ưu:

Công thức $|a - b| = \max(a - b, b - a)$ là "chìa khóa vàng" giúp biến đổi bài toán tìm cực đại chứa dấu trị tuyệt đối thành bài toán tìm cực đại tuyến tính thuần túy.

- **Khoảng cách Euclid 1 chiều:**

$|a - b|$ đo độ dài hình học của đoạn thẳng nối hai điểm a và b trên trục số thực.

2.2. Các Bất đẳng thức Cơ bản

1. Bất đẳng thức tam giác (Triangle Inequality):

$$|a + b| \leq |a| + |b|$$

Dấu bằng xảy ra khi và chỉ khi a và b cùng dấu (tức $a \cdot b \geq 0$).

2. Bất đẳng thức tam giác đảo (Reverse Triangle Inequality):

$$|a - b| \geq ||a| - |b||$$

3. Bất đẳng thức khoảng cách:

Với 3 điểm bất kỳ a, x, b trên trục số:

$$|x - a| + |x - b| \geq |a - b|$$

Dấu bằng xảy ra khi và chỉ khi điểm x nằm **trong đoạn thẳng** nối a và b (giả sử $a \leq b$, khi đó $a \leq x \leq b$).

⚡ 3. Bài toán Tối ưu hóa 1 Chiều: Trung vị (Median Minimization)

3.1. Bài toán Trung vị Cơ bản

Phát biểu bài toán: Cho dãy n số nguyên $a_1, a_2, \dots, a_n$. Hãy tìm một số thực x sao cho tổng khoảng cách từ x đến tất cả các phần tử đạt giá trị nhỏ nhất:

$$F(x) = \sum_{i=1}^n |x - a_i| \rightarrow \min$$

Chứng minh định lý Trung vị:

Giả sử dãy đã được sắp xếp tăng dần: $a_1 \leq a_2 \leq \dots \leq a_n$.

Ta ghép cặp các phần tử đối xứng từ hai đầu: $(a_1, a_n), (a_2, a_{n-1}), \dots, (a_k, a_{n-k+1})$.

Áp dụng bất đẳng thức khoảng cách:

$$|x - a_i| + |x - a_{n-i+1}| \geq a_{n-i+1} - a_i \quad (\forall i = 1, \dots, \lfloor n/2 \rfloor)$$

Dấu bằng đồng thời xảy ra cho cặp thứ i khi và chỉ khi:

$$a_i \leq x \leq a_{n-i+1}$$

Để tổng $F(x) = \sum_{i=1}^{\lfloor n/2 \rfloor} (|x - a_i| + |x - a_{n-i+1}|)$ + (phần tử ở giữa nếu n lẻ) đạt giá trị nhỏ nhất:

- **Trường hợp n lẻ** ($n = 2k + 1$): Điểm x buộc phải thỏa mãn $a_{k+1} \leq x \leq a_{k+1}$, do đó nghiệm duy nhất là **trung vị**:

$$x^* = a_{k+1} = a_{\lfloor n/2 \rfloor + 1}$$

- **Trường hợp n chẵn** ($n = 2k$): Bất kỳ giá trị x nào nằm trong đoạn $[a_k, a_{k+1}]$ đều làm hàm $F(x)$ đạt cực tiểu toàn cục. Trong lập trình thì dấu yêu cầu x nguyên, ta thường chọn:

$$x^* = a_k \quad \text{hoặc} \quad x^* = a_{k+1}$$

Trực quan hóa đoạn giao nhau (Intersection of Intervals):

a1 ----- an	(Tổng = an - a1 khi x thuộc [a1, an])
a2 ----- an-1	(Tổng = an-1 - a2 khi x thuộc [a2, an-1])
...	
[a_k ----- a_{k+1}]	(Vùng tối ưu: Giao của mọi đoạn)

Độ phức tạp thuật toán:

- Cách 1 (`std::sort`): $O(n \log n)$ thời gian, $O(1)$ bộ nhớ phụ.
- Cách 2 (`std::nth_element`): Tìm trung vị trong thời gian tuyến tính $O(n)$, sau đó tính tổng khoảng cách trong $O(n)$. Tổng độ phức tạp tối ưu: $O(n)$.

```
// Tìm trung vị và tính tổng khoảng cách tối ưu O(n)
long long minTotalDistance(std::vector<long long> a) {
    int n = a.size();
    int mid = n / 2;
    std::nth_element(a.begin(), a.begin() + mid, a.end());
    long long median = a[mid];

    long long cost = 0;
    for (long long x : a) {
        cost += std::abs(x - median);
    }
    return cost;
}
```

3.2. Trung vị có Trọng số (Weighted Median)

Phát biểu bài toán: Mỗi điểm a_i có một trọng số (khối lượng/chi phí) $w_i > 0$. Cần tìm x sao cho:

$$F(x) = \sum_{i=1}^n w_i |x - a_i| \rightarrow \min$$

Phương pháp giải:

Đạo hàm theo từng đoạn (hoặc quan sát hệ số góc - slope):

- Khi x tăng từ $-\infty$ đến $+\infty$, đạo hàm của $F(x)$ tại điểm không trùng với a_i là:

$$F'(x) = \sum_{a_i < x} w_i - \sum_{a_i > x} w_i$$

- Điểm cực tiểu xảy ra khi hàm chuyển từ giảm sang tăng, tức là tại điểm a_k sao cho:

$$\sum_{i=1}^{k-1} w_i < \frac{W}{2} \quad \text{và} \quad \sum_{i=1}^k w_i \geq \frac{W}{2}$$

với $W = \sum_{i=1}^n w_i$ là tổng toàn bộ trọng số.

Thuật toán:

1. Sắp xếp các cặp (a_i, w_i) theo a_i tăng dần trong $O(n \log n)$.
2. Tính $W = \sum w_i$.
3. Duyệt mảng tiền tố của trọng số, tìm phần tử đầu tiên có tổng tích lũy $\geq \lceil W/2 \rceil$. Điểm a_k đó chính là **trung vị có trọng số**.

3.3. Kỹ thuật Quy đổi Biến Tuyến tính: $b_i = a_i - i$

Một mô hình kinh điển xuất hiện trong bài toán: Biến đổi dãy số $a_1, a_2, \dots, a_n$ thành một dãy **tăng nghiêm ngặt** ($x_1 < x_2 < \dots < x_n$) sao cho $\sum_{i=1}^n |a_i - x_i|$ đạt cực tiểu.

Phép biến đổi:

$$x_1 < x_2 < \dots < x_n \iff x_1 \leq x_2 - 1 \leq x_3 - 2 \leq \dots \leq x_n - (n-1)$$

Đặt $y_i = x_i - i$ và $b_i = a_i - i$, bài toán tương đương với:

Tìm dãy **không giảm** $y_1 \leq y_2 \leq \dots \leq y_n$ sao cho:

$$\sum_{i=1}^n |b_i - y_i| \rightarrow \min$$

Đây là bài toán nổi tiếng giải được bằng thuật toán **Slope Trick** với hàng đợi ưu tiên (`std::priority_queue`) trong $O(n \log n)$.

4. Khoảng cách Manhattan (L1 Norm) & Hình học Tọa độ

4.1. Tính Độc lập theo Trục Tọa độ (Separability)

Khoảng cách Manhattan giữa hai điểm $P_1(x_1, y_1)$ và $P_2(x_2, y_2)$ trên mặt phẳng $\mathbb{R}^2$ được xác định bởi:

$$d_1(P_1, P_2) = |x_1 - x_2| + |y_1 - y_2|$$

Đặc tính tách biến:

Khi cần tìm một điểm $P^*(x, y)$ để tối ưu hóa hàm tổng khoảng cách Manhattan đến tập điểm $\{P_1, P_2, \dots, P_n\}$:

$$F(x, y) = \sum_{i=1}^n (|x - x_i| + |y - y_i|) = \left(\sum_{i=1}^n |x - x_i| \right) + \left(\sum_{i=1}^n |y - y_i| \right)$$

Do hai biến x và y hoàn toàn độc lập, ta có thể giải quyết hai bài toán trung vị 1D riêng biệt:

- x^* là trung vị của dãy tọa độ $\{x_1, x_2, \dots, x_n\}$.

- y^* là trung vị của dãy tọa độ $\{y_1, y_2, \dots, y_n\}$.

Điểm hợp mặt tối ưu (Best Meeting Point) chính là $P^*(x^*, y^*)$.

4.2. Cặp Điểm Xa nhất theo Khoảng cách Manhattan (Phép biến đổi Chebyshev)

Bài toán: Cho n điểm $P_i(x_i, y_i)$ trên mặt phẳng. Hãy tìm cặp điểm (i, j) có khoảng cách Manhattan lớn nhất:

$$\max_{1 \leq i < j \leq n} (|x_i - x_j| + |y_i - y_j|)$$

Nếu kiểm tra mọi cặp (i, j) , độ phức tạp là $O(n^2)$ (quá chậm khi $n = 2 \cdot 10^5$).

Phép khử dấu trị tuyệt đối:

Vì $|a| = \max(a, -a)$, nên với hai biểu thức $|x_i - x_j|$ và $|y_i - y_j|$, tổng của chúng có đúng 4 khả năng mở dấu:

$$|x_i - x_j| + |y_i - y_j| = \max \begin{cases} +(x_i - x_j) + (y_i - y_j) = (x_i + y_i) - (x_j + y_j) \\ +(x_i - x_j) - (y_i - y_j) = (x_i - y_i) - (x_j - y_j) \\ -(x_i - x_j) + (y_i - y_j) = -(x_i - y_i) + (x_j - y_j) \\ -(x_i - x_j) - (y_i - y_j) = -(x_i + y_i) + (x_j + y_j) \end{cases}$$

Nhận thấy khả năng 3 và 4 chỉ là đảo vị trí i và j của khả năng 2 và 1. Đặt:

$$u_i = x_i + y_i \quad \text{và} \quad v_i = x_i - y_i$$

Khoảng cách Manhattan được viết lại thành **khoảng cách Chebyshev**:

$$d_1(P_i, P_j) = \max(|u_i - u_j|, |v_i - v_j|)$$

Do đó:

$$\max_{i,j} d_1(P_i, P_j) = \max \left(\max_i u_i - \min_j u_j, \max_i v_i - \min_j v_j \right)$$

Thuật toán $O(n)$:

Chỉ cần một vòng lặp duyệt qua n điểm, tìm giá trị lớn nhất và nhỏ nhất của $u_i = x_i + y_i$ và $v_i = x_i - y_i$:

```
long long maxManhattanDistance(const std::vector<std::pair<long long, long long>>& pts) {
    long long max_u = -4e18, min_u = 4e18;
    long long max_v = -4e18, min_v = 4e18;
    for (auto& [x, y] : pts) {
        long long u = x + y;
        long long v = x - y;
        max_u = std::max(max_u, u);
        min_u = std::min(min_u, u);
        max_v = std::max(max_v, v);
        min_v = std::min(min_v, v);
    }
    return std::max(max_u - min_u, max_v - min_v);
}
```

4.3. Khoảng cách Manhattan trong Không gian d Chiều (Kỹ thuật 2^d Mask)

Trong không gian d chiều, khoảng cách Manhattan giữa $A = (a_1, \dots, a_d)$ và $B = (b_1, \dots, b_d)$ là:

$$D(A, B) = \sum_{k=1}^d |a_k - b_k|$$

Mỗi số hạng $|a_k - b_k|$ có thể mang dấu $+$ hoặc $-$. Với d chiều, có tổng cộng 2^d tổ hợp dấu:

$$D(A, B) = \max_{s \in \{-1, +1\}^d} \left(\sum_{k=1}^d s_k a_k - \sum_{k=1}^d s_k b_k \right)$$

Mã hóa vector dấu s bằng một số nguyên $\text{mask} \in [0, 2^d - 1]$, trong đó bit thứ k bằng 1 biểu thị $s_k = +1$, bằng 0 biểu thị $s_k = -1$.

Với mỗi điểm P_i , ta tính giá trị chiếu:

$$\text{val}(i, \text{mask}) = \sum_{k=1}^d (-1)^{(\text{mask} \gg (k-1)) \& 1} \cdot P_{i,k}$$

Khoảng cách cực đại giữa hai điểm trong n điểm là:

$$\max_{i,j} D(P_i, P_j) = \max_{\text{mask}=0}^{2^d-1} \left(\max_i \text{val}(i, \text{mask}) - \min_j \text{val}(j, \text{mask}) \right)$$

- **Độ phức tạp:** $O(n \cdot 2^d)$. Với $d \leq 6$, phương pháp này chạy cực nhanh trong thời gian thực (< 0.1 giây cho $n = 10^5$).



5. Cấu trúc Dữ liệu Động: Duy trì Trung vị (Dynamic Median)

Trong nhiều bài toán CP (như xử lý luồng sự kiện, mảng trượt Sliding Window, hoặc cây truy vấn Segment Tree), ta cần liên tục:

1. Thêm một phần tử v .
2. Xóa một phần tử v .
3. Tìm trung vị hiện tại và tính chi phí $\sum_i |x^* - a_i|$.

Mô hình 2 Heaps (Max-Heap + Min-Heap)

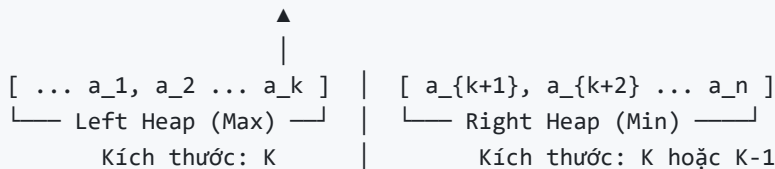
Ta chia tập phần tử hiện tại thành 2 nửa cân bằng:

- `left_heap` (Max-Heap): Chứa nửa các phần tử nhỏ hơn hoặc bằng trung vị.
- `right_heap` (Min-Heap): Chứa nửa các phần tử lớn hơn trung vị.

Đồng thời duy trì hai biến tổng:

- $S_L = \sum_{x \in \text{left}} x$
- $S_R = \sum_{x \in \text{right}} x$

Trung vị (Median = top của Left Heap)



Quy tắc cân bằng:

Luôn duy trì: $|\text{size}(L) - \text{size}(R)| \leq 1$, ưu tiên $\text{size}(L) = \text{size}(R)$ hoặc $\text{size}(L) = \text{size}(R) + 1$.

Khi đó:

- Trung vị tối ưu là $m = \text{top}(L)$.
- Tổng khoảng cách cực tiểu được tính ngay trong $O(1)$:

$$\text{Cost} = (m \cdot \text{size}(L) - S_L) + (S_R - m \cdot \text{size}(R))$$

Kỹ thuật xóa lười (Lazy Deletion):

Trong C++, `std::priority_queue` không hỗ trợ xóa phần tử bất kỳ $O(\log n)$. Ta sử dụng kỹ thuật **Lazy Deletion** bằng một `std::unordered_map<long long, int> dead` để ghi nhận số lần phần tử bị xóa. Khi đỉnh heap rơi vào phần tử đã chết, ta mới `pop()` bỏ.

6. Trace Thuật toán Từng bước (Step-by-Step Execution)

Xét bài toán: Cho dãy tọa độ $A = [7, 1, 9, 3, 5]$. Tìm điểm x nguyên để tối ưu $\sum_{i=1}^5 |x - a_i|$.

Bước 1: Sắp xếp dãy

Dãy sau khi sắp xếp: $A_{\text{sorted}} = [1, 3, 5, 7, 9]$.

Số phần tử $n = 5$ (lẻ).

Bước 2: Ghép cặp đối xứng

- Cặp 1: $(1, 9) \implies |x - 1| + |x - 9| \geq 9 - 1 = 8$ khi $x \in [1, 9]$.
- Cặp 2: $(3, 7) \implies |x - 3| + |x - 7| \geq 7 - 3 = 4$ khi $x \in [3, 7]$.
- Phần tử đơn lẻ ở giữa: $a_3 = 5 \implies |x - 5| \geq 0$ khi $x = 5$.

Bước 3: Tìm giao điểm tối ưu

Giao của các đoạn $[1, 9] \cap [3, 7] \cap \{5\} = \{5\}$.

Vậy trung vị duy nhất là $x^* = 5$.

Bước 4: Tính tổng chi phí

$$\text{Cost} = |5 - 1| + |5 - 3| + |5 - 5| + |5 - 7| + |5 - 9| = 4 + 2 + 0 + 2 + 4 = 12$$

7. Cạm bẫy Phòng thi & Kỹ thuật Tối ưu (Traps & Pitfalls)

1. Tràn số 32-bit (`int` overflow):

2. Với $n = 2 \cdot 10^5$ và $a_i \approx 10^9$, tổng khoảng cách cực tiểu có thể đạt tới:

$$\text{Cost} \approx 2 \cdot 10^5 \times 10^9 = 2 \cdot 10^{14} > 2^{31} - 1$$

3. Luôn sử dụng kiểu dữ liệu `long long` cho các biến tính tổng khoảng cách, mảng tiền tố và tọa độ.

4. Sai lầm khi chọn trung bình cộng (Mean vs. Median):

5. Trung bình cộng $\bar{x} = \frac{1}{n} \sum a_i$ là nghiệm tối ưu của bài toán bình phương khoảng cách $\min \sum (x - a_i)^2$.

6. Đối với giá trị tuyệt đối $\min \sum |x - a_i|$, nghiệm bắt buộc phải là **trung vị (Median)**. Việc dùng trung bình cộng sẽ nhận kết quả Wrong Answer (WA).

7. Bài toán yêu cầu nghiệm x nguyên khi n chẵn:

8. Khi $n = 2k$, mọi số thực trong đoạn $[a_k, a_{k+1}]$ đều cho cùng tổng chi phí.

9. Nếu đề bài yêu cầu thêm ràng buộc phụ (ví dụ: x nhỏ nhất, hoặc bài toán có hệ số), luôn kiểm tra cả hai biên nguyên a_k và a_{k+1} .

10. Hiệu năng tìm kiếm trung vị:

11. Tránh gọi `std::sort` $O(n \log n)$ lặp lại nhiều lần nếu chỉ cần trung vị. Hãy tận dụng `std::nth_element` với độ phức tạp trung bình $O(n)$ để qua mặt các test time limit gắt gao ($< 0.2s$).



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.